

MODUL PRAKTIKUM

Pembuatan Sistem Kasir Kopi Sederhana Berbasis Google Apps Script, Google Sheets, VS Code, Node.js, npm, dan clasp

Modul ini disusun sebagai panduan praktik dari awal sampai akhir untuk membuat Web App kasir kopi sederhana. Pembaca diasumsikan benar-benar pemula, sehingga setiap tahap dijelaskan secara runtut mulai dari instalasi alat, pembuatan database, penulisan kode, push project menggunakan clasp, deployment Web App, hingga update deployment versi baru.

Studi kasus yang digunakan adalah Sistem Kasir Kopi Sederhana. Google Sheets digunakan sebagai database, Google Apps Script digunakan sebagai backend, sedangkan HTML, CSS, dan JavaScript digunakan sebagai frontend. Aplikasi berjalan di server Google Apps Script, bukan di localhost. Laragon dan Docker tidak diperlukan.

1. Gambaran Umum

Sistem kasir kopi sederhana ini digunakan untuk menampilkan daftar produk kopi, memilih produk, menghitung total belanja, memasukkan nominal bayar, menghitung kembalian, serta menyimpan transaksi ke Google Sheets. Project dibuat secara lokal di VS Code, lalu dikirim ke Google Apps Script menggunakan clasp.

2. Tujuan Pembelajaran

- Menginstall Node.js dan mengecek node serta npm.
- Mengatasi error PowerShell running scripts is disabled on this system.
- Menginstall dan login clasp.
- Mengaktifkan Google Apps Script API.
- Membuat project Google Apps Script dari VS Code.
- Membuat Google Sheets sebagai database.
- Membuat backend Apps Script dan frontend Web App.
- Melakukan clasp push, authorization, deploy, testing, dan update deployment.

3. Tools yang Dibutuhkan

Tools	Fungsi
Node.js	Runtime JavaScript dan penyedia npm.
npm	Package manager untuk menginstall clasp.
clasp	Command line tool untuk mengelola project Google Apps Script dari lokal.
VS Code	Editor kode.
Google Apps Script	Backend dan hosting Web App.
Google Sheets	Database sederhana untuk produk dan transaksi.

4. Konsep Dasar Arsitektur Sistem

Arsitektur sistem terdiri dari tiga bagian utama. Pertama, frontend berupa file index.html, style.html, dan script.html yang menampilkan antarmuka kasir. Kedua, backend berupa file Code.js, ProductService.js, dan TransaksiService.js yang berjalan di Google Apps Script. Ketiga, database berupa Google Sheets dengan sheet Produk, Transaksi, dan DetailTransaksi.

Alur kerja aplikasi: pengguna membuka URL Web App, Apps Script menjalankan doGet(), halaman kasir tampil, frontend memanggil fungsi backend getProduk(), data produk dibaca dari Google Sheets, pengguna membuat transaksi, lalu data transaksi disimpan kembali ke Google Sheets.

5. Instalasi Node.js

Buka situs resmi Node.js, unduh versi LTS, lalu install seperti aplikasi biasa. Centang opsi yang direkomendasikan selama instalasi. Setelah selesai, tutup dan buka kembali terminal atau PowerShell agar perintah node dan npm dikenali.

6. Pengecekan node dan npm

```
node -v
npm -v
```

Jika muncul nomor versi, berarti Node.js dan npm sudah berhasil terpasang.

7. Solusi Error npm di PowerShell

Jika muncul error running scripts is disabled on this system, jalankan PowerShell sebagai Administrator, lalu ketik:

```
Set-ExecutionPolicy RemoteSigned
```

Pilih Y lalu Enter. Setelah itu tutup PowerShell dan buka kembali. Jika tidak ingin mengubah policy global, gunakan Command Prompt atau terminal Git Bash sebagai alternatif.

8. Instalasi clasp

```
npm install -g @google/clasp  
clasp -v
```

Jika versi clasp muncul, instalasi berhasil.

9. Login clasp

```
clasp login
```

Browser akan terbuka. Login menggunakan akun Google yang akan dipakai untuk Apps Script dan Google Sheets. Berikan izin yang diminta.

10. Aktivasi Google Apps Script API

Buka halaman Google Apps Script API melalui browser, lalu aktifkan API pada akun Google yang sama. Tahap ini penting agar clasp dapat membuat dan mengelola project Apps Script.

11. Membuat Folder Project Lokal di VS Code

```
mkdir kasir-kopi-gas  
cd kasir-kopi-gas  
code .
```

Perintah tersebut membuat folder project dan membukanya di VS Code.

12. Membuat Project Apps Script dengan clasp

```
clasp create --type webapp --title "Kasir Kopi GAS"
```

Perintah ini membuat project Apps Script baru bertipe Web App. Jika muncul pilihan, pilih standalone project.

13. Penjelasan .clasp.json dan appsscript.json

File .clasp.json menyimpan scriptId project Apps Script. File ini menghubungkan folder lokal dengan project Apps Script di akun Google. File appsscript.json adalah manifest Apps Script yang berisi konfigurasi runtime, timezone, dan pengaturan Web App.

14. Membuat Google Sheets Database

Buat Google Sheets baru dengan nama Database Kasir Kopi. Di dalamnya buat tiga sheet: Produk, Transaksi, dan DetailTransaksi.

Sheet Produk

id_produk	nama_produk	harga	stok
-----------	-------------	-------	------

P001	Kopi Hitam	5000	50
P002	Kopi Susu	7000	40
P003	Americano	12000	30
P004	Cappuccino	15000	25
P005	Latte	18000	20

Sheet Transaksi

Gunakan header: id_transaksi | tanggal | total | bayar | kembalian

Sheet DetailTransaksi

Gunakan header: id_transaksi | id_produk | nama_produk | harga | qty | subtotal

15. Mengambil Spreadsheet ID

Spreadsheet ID terdapat pada URL Google Sheets. Contoh URL: https://docs.google.com/spreadsheets/d/SPREADSHEET_ID/edit. Salin bagian di antara /d/ dan /edit.

16. Struktur File Project

```
kasir-kopi-gas/
■■■■ .clasp.json
■■■■ appsscript.json
■■■■ Code.js
■■■■ ProductService.js
■■■■ TransaksiService.js
■■■■ index.html
■■■■ style.html
■■■■ script.html
```

17. Kode appsscript.json

```
{
  "timeZone": "Asia/Jakarta",
  "dependencies": {},
  "exceptionLogging": "STACKDRIVER",
  "runtimeVersion": "V8",
  "webapp": {
    "executeAs": "USER_DEPLOYING",
    "access": "ANYONE"
  }
}
```

18. Kode Code.js

```
const SPREADSHEET_ID = "MASUKKAN_SPREADSHEET_ID_KAMU";

function doGet() {
  return HtmlService.createTemplateFromFile("index")
    .evaluate()
    .setTitle("Kasir Kopi")
    .setXFrameOptionsMode(HtmlService.XFrameOptionsMode.ALLOWALL);
}

function include(filename) {
  return HtmlService.createHtmlOutputFromFile(filename).getContent();
}

function getSpreadsheet() {
  return SpreadsheetApp.openById(SPREADSHEET_ID);
}
```

19. Kode ProductService.js

```
function getProduk() {
```

```

const ss = getSpreadsheet();
const sheet = ss.getSheetByName("Produk");
if (!sheet) {
  throw new Error("Sheet Produk tidak ditemukan");
}

const values = sheet.getDataRange().getValues();
const headers = values.shift();

return values.map(row => {
  return {
    id_produk: row[0],
    nama_produk: row[1],
    harga: Number(row[2]),
    stok: Number(row[3])
  };
});
}

```

20. Kode TransaksiService.js

```

function simpanTransaksi(data) {
  const ss = getSpreadsheet();
  const sheetTransaksi = ss.getSheetByName("Transaksi");
  const sheetDetail = ss.getSheetByName("DetailTransaksi");

  if (!sheetTransaksi || !sheetDetail) {
    throw new Error("Sheet Transaksi atau DetailTransaksi tidak ditemukan");
  }

  const idTransaksi = "TRX" + new Date().getTime();
  const tanggal = new Date();

  sheetTransaksi.appendRow([
    idTransaksi,
    tanggal,
    data.total,
    data.bayar,
    data.kembalian
  ]);

  data.items.forEach(item => {
    sheetDetail.appendRow([
      idTransaksi,
      item.id_produk,
      item.nama_produk,
      item.harga,
      item.qty,
      item.subtotal
    ]);
  });

  return {
    status: true,
    message: "Transaksi berhasil disimpan",
    id_transaksi: idTransaksi
  };
}

```

21. Kode index.html

```

<!DOCTYPE html>
<html>
<head>
  <base target="_top">
  <?!= include("style"); ?>
</head>
<body>
  <div class="container">
    <h1>Kasir Kopi</h1>
    <p class="subtitle">Sistem Kasir Kopi Sederhana Berbasis Google Apps Script</p>

    <div class="layout">
      <div class="card">
        <h2>Daftar Produk</h2>
        <div id="produkList">Memuat produk...</div>
      </div>

      <div class="card">
        <h2>Keranjang</h2>
        <table>
          <thead>

```

```

        <tr>
            <th>Produk</th>
            <th>Qty</th>
            <th>Subtotal</th>
            <th>Aksi</th>
        </tr>
    </thead>
    <tbody id="keranjangBody"></tbody>
</table>

<div class="summary">
    <p>Total: <strong id="totalText">Rp0</strong></p>
    <label>Bayar</label>
    <input type="number" id="bayarInput" placeholder="Masukkan nominal bayar"
oninput="hitungKembalian()">
    <p>Kembalian: <strong id="kembalianText">Rp0</strong></p>
    <button onclick="simpan()">Simpan Transaksi</button>
</div>
</div>
</div>
</div>

<?!= include("script"); ?>
</body>
</html>

```

22. Kode style.html

```

<style>
body {
    font-family: Arial, sans-serif;
    background: #f4f1ea;
    margin: 0;
    padding: 0;
    color: #2f241d;
}

.container {
    max-width: 1100px;
    margin: auto;
    padding: 24px;
}

h1 {
    text-align: center;
    margin-bottom: 4px;
}

.subtitle {
    text-align: center;
    margin-bottom: 24px;
}

.layout {
    display: grid;
    grid-template-columns: 1fr 1.3fr;
    gap: 20px;
}

.card {
    background: #ffffff;
    border-radius: 12px;
    padding: 18px;
    box-shadow: 0 4px 12px rgba(0,0,0,0.08);
}

.produk-item {
    border: 1px solid #ddd;
    border-radius: 10px;
    padding: 12px;
    margin-bottom: 10px;
    display: flex;
    justify-content: space-between;
    align-items: center;
}

button {
    background: #6f4e37;
    color: white;
    border: none;
    padding: 9px 13px;
    border-radius: 8px;
    cursor: pointer;
}

```

```

}

button:hover {
  background: #563927;
}

table {
  width: 100%;
  border-collapse: collapse;
}

th, td {
  border-bottom: 1px solid #ddd;
  padding: 8px;
  text-align: left;
}

input {
  width: 100%;
  padding: 10px;
  margin-top: 6px;
  box-sizing: border-box;
}

.summary {
  margin-top: 16px;
}
</style>

```

23. Kode script.html

```

<script>
let produk = [];
let keranjang = [];

function formatRupiah(angka) {
  return "Rp" + Number(angka).toLocaleString("id-ID");
}

function loadProduk() {
  google.script.run
    .withSuccessHandler(function(data) {
      produk = data;
      renderProduk();
    })
    .withFailureHandler(function(error) {
      document.getElementById("produkList").innerHTML = error.message;
    })
    .getProduk();
}

function renderProduk() {
  const produkList = document.getElementById("produkList");
  produkList.innerHTML = "";

  produk.forEach(item => {
    const div = document.createElement("div");
    div.className = "produk-item";
    div.innerHTML = `
      <div>
        <strong>${item.nama_produk}</strong><br>
        ${formatRupiah(item.harga)} | Stok: ${item.stok}
      </div>
      <button onclick="tambahKeranjang('${item.id_produk}')">Tambah</button>
    `;
    produkList.appendChild(div);
  });
}

function tambahKeranjang(idProduk) {
  const item = produk.find(p => p.id_produk === idProduk);
  const existing = keranjang.find(k => k.id_produk === idProduk);

  if (existing) {
    existing.qty++;
    existing.subtotal = existing.qty * existing.harga;
  } else {
    keranjang.push({
      id_produk: item.id_produk,
      nama_produk: item.nama_produk,
      harga: item.harga,
      qty: 1,
      subtotal: item.harga
    });
  }
}

```

```

    });
  }

  renderKeranjang();
}

function renderKeranjang() {
  const body = document.getElementById("keranjangBody");
  body.innerHTML = "";

  keranjang.forEach((item, index) => {
    const tr = document.createElement("tr");
    tr.innerHTML = `
      <td>${item.nama_produk}</td>
      <td>${item.qty}</td>
      <td>${formatRupiah(item.subtotal)}</td>
      <td><button onclick="hapusItem(${index})">Hapus</button></td>
    `;
    body.appendChild(tr);
  });

  hitungTotal();
}

function hapusItem(index) {
  keranjang.splice(index, 1);
  renderKeranjang();
}

function hitungTotal() {
  const total = keranjang.reduce((sum, item) => sum + item.subtotal, 0);
  document.getElementById("totalText").innerText = formatRupiah(total);
  hitungKembalian();
}

function hitungKembalian() {
  const total = keranjang.reduce((sum, item) => sum + item.subtotal, 0);
  const bayar = Number(document.getElementById("bayarInput").value || 0);
  const kembalian = bayar - total;
  document.getElementById("kembalianText").innerText = formatRupiah(kembalian < 0 ? 0 : kembalian);
}

function simpan() {
  const total = keranjang.reduce((sum, item) => sum + item.subtotal, 0);
  const bayar = Number(document.getElementById("bayarInput").value || 0);
  const kembalian = bayar - total;

  if (keranjang.length === 0) {
    alert("Keranjang masih kosong");
    return;
  }

  if (bayar < total) {
    alert("Nominal bayar kurang");
    return;
  }

  const data = {
    items: keranjang,
    total: total,
    bayar: bayar,
    kembalian: kembalian
  };

  google.script.run
    .withSuccessHandler(function(res) {
      alert(res.message + "\nID: " + res.id_transaksi);
      keranjang = [];
      document.getElementById("bayarInput").value = "";
      renderKeranjang();
    })
    .withFailureHandler(function(error) {
      alert(error.message);
    })
    .simpanTransaksi(data);
}

loadProduk();
</script>

```

24. Push Project ke Apps Script


```
clasp push
```

Jika muncul pertanyaan untuk overwrite file, pilih Yes. Setelah push berhasil, kode lokal sudah masuk ke project Apps Script.

25. Membuka Project Apps Script

Jika perintah clasp open tersedia, jalankan:

```
clasp open
```

Jika clasp open tidak dikenal, gunakan alternatif berikut:

```
clasp list
```

Salin URL atau scriptId yang muncul, lalu buka project Apps Script melalui browser.

26. Menjalankan Function getProduk dan Authorization

Di editor Apps Script, pilih function getProduk, lalu klik Run. Google akan meminta authorization. Pilih akun, klik Advanced jika diperlukan, lalu izinkan akses. Authorization diperlukan karena script membaca Google Sheets.

27. Deploy sebagai Web App

Klik Deploy > New deployment. Pilih type Web app. Isi deskripsi versi, misalnya Versi 1. Pada bagian Execute as pilih Me. Pada bagian Who has access pilih Anyone atau Anyone with Google account sesuai kebutuhan. Klik Deploy, lalu salin URL Web App.

URL /exec adalah URL production. URL /dev adalah URL pengujian yang biasanya hanya bisa diakses oleh pemilik atau editor script.

28. Menguji Aplikasi

Buka URL /exec. Pastikan daftar produk tampil. Tambahkan produk ke keranjang, masukkan nominal bayar, lalu klik Simpan Transaksi. Cek sheet Transaksi dan DetailTransaksi untuk memastikan data tersimpan.

29. Cara Update Kode

Setiap perubahan kode dilakukan di VS Code. Setelah selesai, jalankan:

```
clasp push
```

Namun, untuk Web App production, clasp push saja belum cukup. Deployment production perlu diarahkan ke versi baru.

30. Cara Edit Deployment / Membuat Versi Baru

Buka Apps Script, klik Deploy > Manage deployments. Pilih deployment Web App yang sudah ada, klik Edit, pilih New version, isi deskripsi versi, lalu klik Deploy. Dengan cara ini URL Web App tetap sama, tetapi kode production diperbarui.

Jika membuat deployment baru dari awal, URL Web App dapat berubah. Jika ingin URL tetap sama, gunakan Manage deployments dan edit deployment lama.

31. Troubleshooting

npm tidak bisa jalan karena PowerShell Execution Policy

Jalankan PowerShell sebagai Administrator lalu gunakan Set-ExecutionPolicy RemoteSigned, atau gunakan Command Prompt/Git Bash.

Project file already exists

Folder sudah berisi file project. Gunakan folder kosong, hapus file yang bentrok, atau gunakan clasp clone jika ingin mengambil project yang sudah ada.

clasp open tidak dikenal

Gunakan clasp list untuk melihat daftar project, lalu buka project Apps Script secara manual melalui browser.

Sheet Produk tidak ditemukan

Pastikan nama sheet persis Produk, tanpa spasi tambahan dan huruf besar kecil sesuai.

Permission denied ke Google Sheets

Pastikan akun yang menjalankan Apps Script memiliki akses ke spreadsheet dan SPREADSHEET_ID benar.

Produk tidak muncul di Web App

Cek function getProduk, cek nama sheet, cek authorization, dan lihat Executions di Apps Script.

Perubahan kode tidak muncul setelah deploy

Jalankan clasp push, lalu edit deployment dan pilih New version.

Script function not found: doGet

Pastikan file Code.js memiliki function doGet() dan sudah berhasil di-push.

Cannot read properties of null

Biasanya terjadi karena getSheetByName menghasilkan null. Periksa nama sheet atau elemen HTML yang dipanggil JavaScript.

32. Rekomendasi Pengembangan Lanjutan

- Menambahkan login kasir.
- Menambahkan cetak struk.
- Menambahkan laporan penjualan harian.
- Menambahkan pengurangan stok otomatis.
- Menambahkan dashboard grafik penjualan.
- Menambahkan validasi input yang lebih lengkap.

33. Kesimpulan

Dengan modul ini, pembaca dapat membuat sistem kasir kopi sederhana dari nol menggunakan Google Apps Script, Google Sheets, VS Code, Node.js, npm, dan clasp. Project dikembangkan secara lokal, dikirim ke Apps Script menggunakan clasp push, lalu dijalankan sebagai Web App di server Google Apps Script. Untuk update production, perubahan kode harus di-push dan deployment harus diperbarui menggunakan versi baru agar URL Web App tetap sama.